Objective:- Predict the possibility of diseases based on patient data.

Importing the dependices

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler,LabelEncoder
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
import sklearn.datasets
from sklearn.metrics import (accuracy_score,confusion_matrix,classification_report,precision_score,recall_score,f1_score,roc_auc_
```

```
breast_cancer_dataset = sklearn.datasets.load_breast_cancer()
```

```
print(breast_cancer_dataset)
```

```
{'data': array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01, 4.601e-01,
        1.189e-01],
       [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
        8.902e-02],
       [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,
        8.758e-02],
       ...,
       [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
        7.820e-02],
       [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
        1.240e-01],
       [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
        7.039e-02]]), 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
       1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,
       1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
       1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
       0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1,
       1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0,
       0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0,
       0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
       1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1,
       1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1,
       1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1,
       1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1]), 'frame': None, 'target_names': array(['malignant', 'benign'],
      'mean smoothness', 'mean compactness', 'mean concavity',
      'mean concave points', 'mean symmetry', 'mean fractal dimension',
      'radius error', 'texture error', 'perimeter error', 'area error',
      'smoothness error', 'compactness error', 'concavity error',
      'concave points error', 'symmetry error',
      'fractal dimension error', 'worst radius', 'worst texture',
      'worst perimeter', 'worst area', 'worst smoothness',
      'worst compactness', 'worst concavity', 'worst concave points',
      'worst symmetry', 'worst fractal dimension'], dtype='<U23'), 'filename': 'breast_cancer.csv', 'data_module': 'sklearn.dat
```

```
breast_cancer_dataset = pd.DataFrame(breast_cancer_dataset.data, columns=breast_cancer_dataset.feature_names)
```

```
breast_cancer_dataset.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	per
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.50	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.67	

5 rows × 30 columns

```
breast_cancer_dataset.tail()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	p
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	0.1726	0.05623	...	25.450	26.40	
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	0.1752	0.05533	...	23.690	38.25	
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	0.1590	0.05648	...	18.980	34.12	
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	0.2397	0.07016	...	25.740	39.42	
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	0.1587	0.05884	...	9.456	30.37	

5 rows × 30 columns

```
breast_cancer_dataset.shape
```

```
(569, 30)
```

```
breast_cancer_dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 30 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   mean radius              569 non-null    float64
 1   mean texture             569 non-null    float64
 2   mean perimeter           569 non-null    float64
 3   mean area                569 non-null    float64
 4   mean smoothness          569 non-null    float64
 5   mean compactness         569 non-null    float64
 6   mean concavity           569 non-null    float64
 7   mean concave points      569 non-null    float64
 8   mean symmetry            569 non-null    float64
 9   mean fractal dimension   569 non-null    float64
 10  radius error             569 non-null    float64
 11  texture error            569 non-null    float64
 12  perimeter error          569 non-null    float64
 13  area error               569 non-null    float64
 14  smoothness error         569 non-null    float64
 15  compactness error        569 non-null    float64
 16  concavity error          569 non-null    float64
 17  concave points error     569 non-null    float64
 18  symmetry error           569 non-null    float64
 19  fractal dimension error  569 non-null    float64
 20  worst radius             569 non-null    float64
 21  worst texture            569 non-null    float64
 22  worst perimeter          569 non-null    float64
 23  worst area               569 non-null    float64
 24  worst smoothness         569 non-null    float64
 25  worst compactness        569 non-null    float64
 26  worst concavity          569 non-null    float64
 27  worst concave points     569 non-null    float64
 28  worst symmetry           569 non-null    float64
 29  worst fractal dimension  569 non-null    float64
dtypes: float64(30)
memory usage: 133.5 KB
```

```
breast_cancer_dataset.describe()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	...
mean	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	0.062798	...
std	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	0.007060	...
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	0.049960	...
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	0.057700	...
50%	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	0.061540	...
75%	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	0.066120	...
max	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	0.097440	...

8 rows × 30 columns

```
breast_cancer_dataset.isnull().sum()
```

	0
mean radius	0
mean texture	0
mean perimeter	0
mean area	0
mean smoothness	0
mean compactness	0
mean concavity	0
mean concave points	0
mean symmetry	0
mean fractal dimension	0
radius error	0
texture error	0
perimeter error	0
area error	0
smoothness error	0
compactness error	0
concavity error	0
concave points error	0
symmetry error	0
fractal dimension error	0
worst radius	0
worst texture	0
worst perimeter	0
worst area	0
worst smoothness	0
worst compactness	0
worst concavity	0
worst concave points	0
worst symmetry	0
worst fractal dimension	0

dtype: int64

dtype: int64

```
breast_cancer_dataset['mean texture'].value_counts()
```

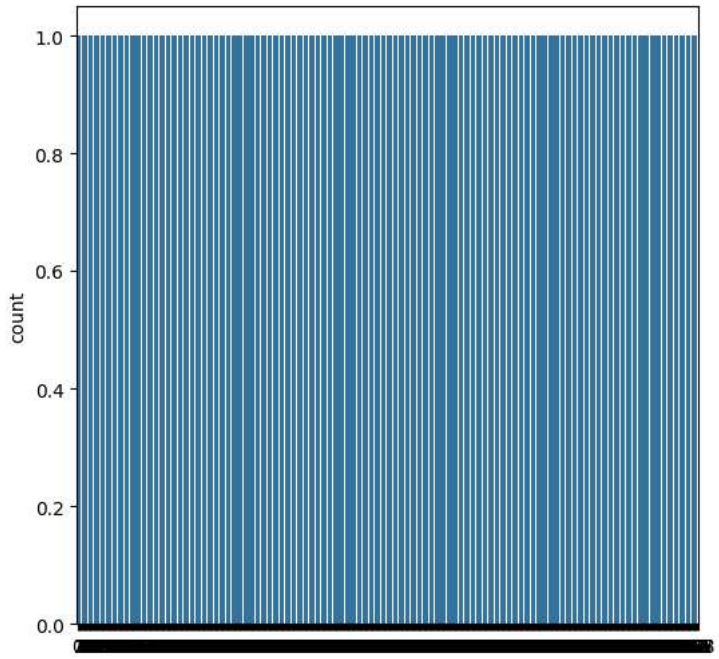
mean texture	count
16.84	3
19.83	3
15.70	3
20.52	3
18.22	3
...	...
27.88	1
22.68	1
23.93	1
29.37	1
30.62	1

479 rows × 1 columns

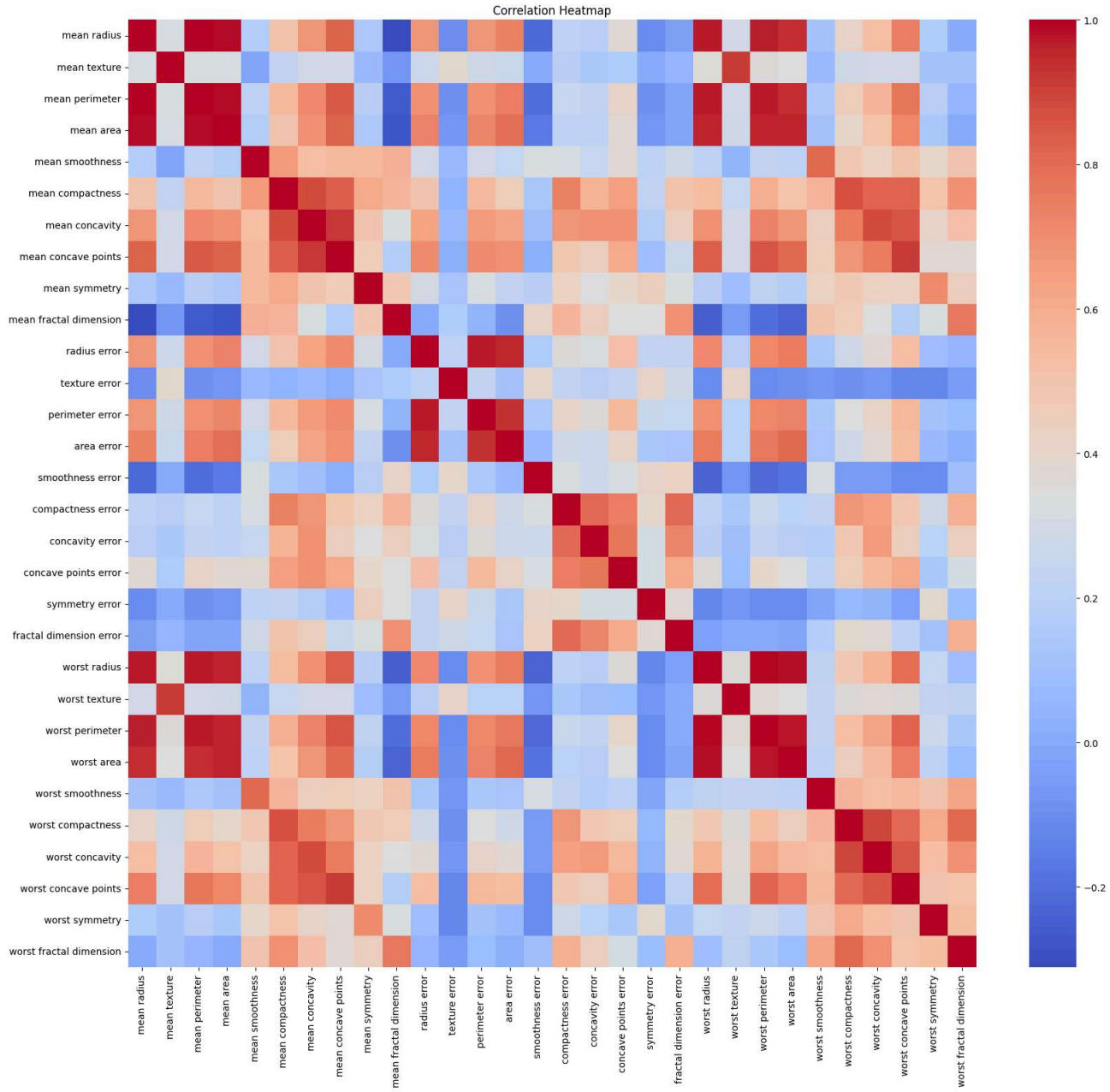
dtype: int64

Data Visualizations

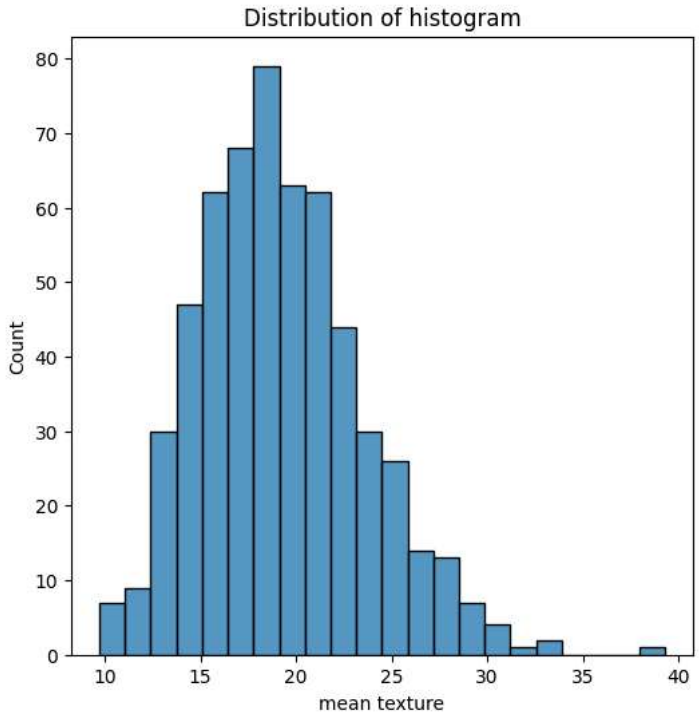
```
#countplot
plt.figure(figsize=(6,6))
sns.countplot(breast_cancer_dataset['mean texture'])
plt.show()
```



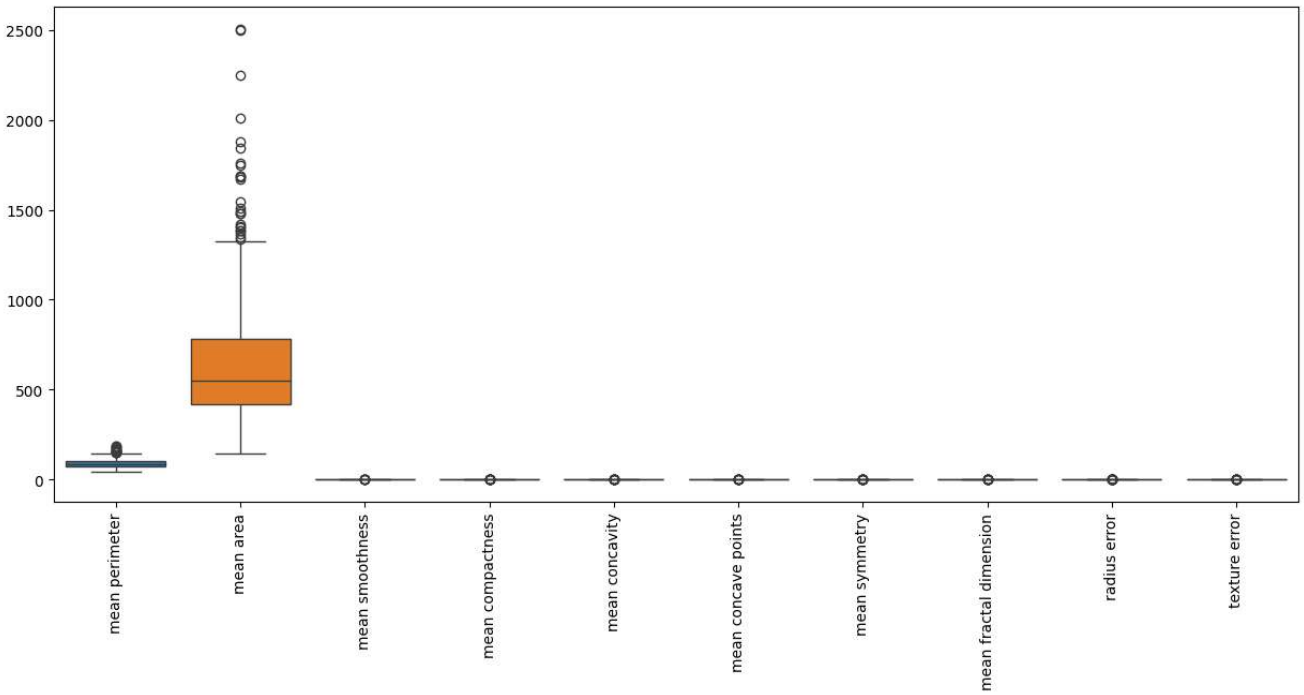

```
#heat map
plt.figure(figsize=(20,18))
sns.heatmap(breast_cancer_dataset.corr(numeric_only=True), cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```




```
#histogram
plt.figure(figsize=(6,6))
sns.histplot(breast_cancer_dataset['mean texture'])
plt.title("Distribution of histogram")
plt.show()
```

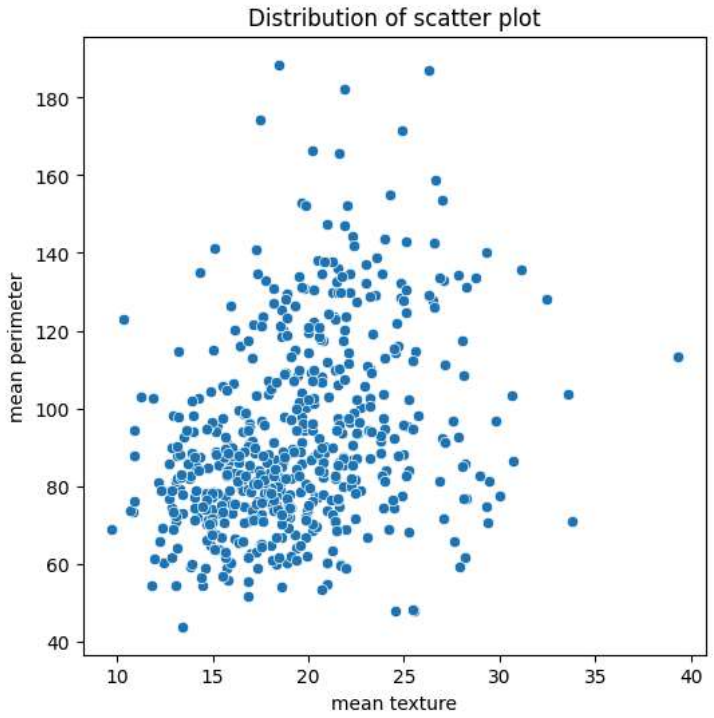



```
#box plot
plt.figure(figsize=(15,6))
sns.boxplot(data=breast_cancer_dataset.iloc[:,2:12])
plt.xticks(rotation=90)
plt.show()
```




```
#scatter plot
plt.figure(figsize=(6,6))
sns.scatterplot(x='mean texture',y='mean perimeter',data=breast_cancer_dataset)
```

```
plt.title("Distribution of scatter plot")
plt.show()
```




```
print(breast_cancer_dataset.columns)
```

```
Index(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
       'mean smoothness', 'mean compactness', 'mean concavity',
       'mean concave points', 'mean symmetry', 'mean fractal dimension',
       'radius error', 'texture error', 'perimeter error', 'area error',
       'smoothness error', 'compactness error', 'concavity error',
       'concave points error', 'symmetry error', 'fractal dimension error',
       'worst radius', 'worst texture', 'worst perimeter', 'worst area',
       'worst smoothness', 'worst compactness', 'worst concavity',
       'worst concave points', 'worst symmetry', 'worst fractal dimension'],
      dtype='object')
```

```
#adding label column
breast_cancer_dataset['label'] = breast_cancer_dataset['mean symmetry']
```

```
breast_cancer_dataset.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.60
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.80
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.50
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.87
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.20

5 rows × 31 columns

```
breast_cancer_dataset['label'].value_counts()
```

label	count
0.1601	4
0.1769	4
0.1717	4
0.1893	4
0.1714	4
...	...
0.2384	1
0.1703	1
0.1167	1
0.2395	1
0.2251	1

432 rows × 1 columns

dtype: int64

```
X = breast_cancer_dataset.drop(columns='label',axis=1)
Y = breast_cancer_dataset['label']
```

```
print(X)
```

```
     mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0          17.99         10.38          122.80     1001.0          0.11840
1          20.57         17.77          132.90     1326.0          0.08474
2          19.69         21.25          130.00     1203.0          0.10960
3          11.42         20.38           77.58      386.1          0.14250
4          20.29         14.34          135.10     1297.0          0.10030
..           ...           ...             ...        ...              ...
564        21.56         22.39          142.00     1479.0          0.11100
565        20.13         28.25          131.20     1261.0          0.09780
566        16.60         28.08          108.30      858.1          0.08455
567        20.60         29.33          140.10     1265.0          0.11780
568         7.76         24.54           47.92      181.0          0.05263

     mean compactness  mean concavity  mean concave points  mean symmetry  \
0             0.27760         0.30010              0.14710         0.2419
1             0.07864         0.08690              0.07017         0.1812
2             0.15990         0.19740              0.12790         0.2069
3             0.28390         0.24140              0.10520         0.2597
4             0.13280         0.19800              0.10430         0.1809
..                ...             ...                  ...            ...
564           0.11590         0.24390              0.13890         0.1726
565           0.10340         0.14400              0.09791         0.1752
566           0.10230         0.09251              0.05302         0.1590
567           0.27700         0.35140              0.15200         0.2397
568           0.04362         0.00000              0.00000         0.1587

     mean fractal dimension  ...  worst radius  worst texture  \
0                   0.07871  ...        25.380          17.33
1                   0.05667  ...        24.990          23.41
2                   0.05999  ...        23.570          25.53
3                   0.09744  ...        14.910          26.50
4                   0.05883  ...        22.540          16.67
..                      ...  ...           ...            ...
564                 0.05623  ...        25.450          26.40
565                 0.05533  ...        23.690          38.25
566                 0.05648  ...        18.980          34.12
567                 0.07016  ...        25.740          39.42
568                 0.05884  ...         9.456          30.37

     worst perimeter  worst area  worst smoothness  worst compactness  \
0             184.60      2019.0           0.16220            0.66560
1             158.80      1956.0           0.12380            0.18660
2             152.50      1709.0           0.14440            0.42450
3              98.87       567.7           0.20980            0.86630
4             152.20      1575.0           0.13740            0.20500
..               ...         ...               ...                ...
564           166.10      2027.0           0.14100            0.21130
565           155.00      1731.0           0.11660            0.19220
566           126.70      1124.0           0.11390            0.30940
```

```
567          184.60      1821.0           0.16500            0.86810
568           59.16       268.6           0.08996            0.06444

     worst concavity  worst concave points  worst symmetry  \
0             0.7119                0.2654          0.4601
1             0.2416                0.1860          0.2750
2             0.4504                0.2430          0.3613
3             0.6869                0.2575          0.6638
4             0.4000                0.1625          0.2364
```

```
import sklearn.datasets

breast_cancer_dataset = sklearn.datasets.load_breast_cancer()

X = breast_cancer_dataset.data
Y = breast_cancer_dataset.target
```

Spliting the data into the training and testing

```
X_train,X_test,Y_train,Y_test = train_test_split(X,Y,test_size=0.2,random_state=2)
```

```
print(X_train.shape,X_test.shape)
```

```
(455, 30) (114, 30)
```

```
print(X_train)
```

```
[[1.405e+01 2.715e+01 9.138e+01 ... 1.048e-01 2.250e-01 8.321e-02]
 [1.113e+01 1.662e+01 7.047e+01 ... 4.044e-02 2.383e-01 7.083e-02]
 [1.918e+01 2.249e+01 1.275e+02 ... 1.708e-01 3.193e-01 9.221e-02]
 ...
 [1.246e+01 1.283e+01 7.883e+01 ... 2.680e-02 2.280e-01 7.028e-02]
 [1.234e+01 1.227e+01 7.894e+01 ... 1.070e-01 3.110e-01 7.592e-02]
 [1.747e+01 2.468e+01 1.161e+02 ... 1.721e-01 2.160e-01 9.300e-02]]
```

```
print(X_test)
```

```
[[1.394e+01 1.317e+01 9.031e+01 ... 1.015e-01 2.160e-01 7.253e-02]
 [1.496e+01 1.910e+01 9.703e+01 ... 1.489e-01 2.962e-01 8.472e-02]
 [9.668e+00 1.810e+01 6.106e+01 ... 2.500e-02 3.057e-01 7.875e-02]
 ...
 [1.665e+01 2.138e+01 1.100e+02 ... 2.095e-01 3.613e-01 9.564e-02]
 [1.499e+01 2.520e+01 9.554e+01 ... 2.899e-02 1.565e-01 5.504e-02]
 [1.705e+01 1.908e+01 1.134e+02 ... 2.543e-01 3.109e-01 9.061e-02]]
```

```
print(Y_train)
```

```
[1 1 0 0 0 1 1 0 1 1 0 0 1 0 1 1 0 0 1 0 0 1 1 1 1 1 1 1 1 0 0 1 1 1 0 1 1
 0 0 0 1 1 0 1 1 1 1 0 0 1 1 1 0 0 1 1 1 1 1 1 0 1 0 1 0 1 1 1 0 1 1 1 0 0
 1 0 0 1 1 0 1 1 1 1 1 1 1 1 0 1 1 1 1 1 0 1 0 1 1 0 1 1 0 0 1 0 0 0 1 0 1
 1 0 0 0 1 0 1 1 0 1 0 1 0 1 1 1 1 0 0 0 1 1 0 1 1 1 1 1 1 1 0 1 1 1 0 1 0 0 0
 1 1 0 0 1 1 0 1 1 1 1 1 0 0 1 1 1 0 1 1 1 0 1 0 1 1 1 1 1 1 1 0 0 1 1 0 1 1 0
 0 0 1 1 0 1 0 1 0 0 0 0 1 1 0 1 0 1 1 0 1 1 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0
 1 1 0 0 1 1 0 1 1 1 0 0 1 1 0 0 0 0 1 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 0 1 0 1
 1 1 1 1 1 0 1 1 0 1 1 1 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 1 1
 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 0 0 1 1 1 1
 0 0 0 1 0 1 1 1 1 1 1 0 0 0 1 0 1 0 1 1 1 1 0 0 1 0 0 1 1 1 1 1 1 1 0 1 1 1 1
 1 1 1 0 1 1 1 0 1 1 0 1 0 1 0 1 0 1 1 1 1 1 0 1 1 1 1 1 0 0 0 1 1 0 1 0 0 0 1
 0 0 1 0 1 1 0 1 1 1 1 0 1 0 1 0 1 1 1 1 0 0 1 0 1 0 1 1 1 0 0 1 1 1 1 1 1 1 1 1
 0 0 0 1 0 1 1 1 1 1 0]
```

```
print(Y_train)
print(Y_train.dtype)
print(set(Y_train))
```

```
[1 1 0 0 0 1 1 0 1 1 0 0 1 0 1 1 0 0 1 0 0 1 1 1 1 1 1 1 1 0 0 1 1 1 0 1 1
 0 0 0 1 1 0 1 1 1 1 0 0 1 1 1 0 0 1 1 1 1 1 1 0 1 0 1 0 1 1 1 0 1 1 1 0 0
 1 0 0 1 1 0 1 1 1 1 1 1 1 1 0 1 1 1 1 1 0 1 0 1 1 0 1 1 0 0 1 0 0 0 1 0 1
 1 0 0 0 1 0 1 1 0 1 0 1 0 1 1 1 1 0 0 0 1 1 0 1 1 1 1 1 1 1 0 1 1 1 0 1 0 0 0
 1 1 0 0 1 1 0 1 1 1 1 1 0 0 1 1 1 0 1 1 1 0 1 0 1 1 1 1 1 1 1 0 0 1 1 0 1 1 0
 0 0 1 1 0 1 0 1 0 0 0 0 1 1 0 1 0 1 1 0 1 1 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0
 1 1 0 0 1 1 0 1 1 1 0 0 1 1 0 0 0 0 1 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 0 1 0 1
 1 1 1 1 1 0 1 1 0 1 1 1 1 1 0 1 1 0 1 0 0 1 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1
 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 0 0 1 1 1 1
 0 0 0 1 0 1 1 1 1 1 1 0 0 0 1 0 1 0 1 1 1 1 0 0 1 0 0 1 1 1 1 1 1 1 0 1 1 1 1 0
 1 1 1 0 1 1 1 0 1 1 0 1 0 1 0 1 1 1 1 0 0 1 1 0 1 0 1 1 1 0 0 0 1 1 0 1 0 0 0 1
 0 0 1 0 1 1 0 1 1 1 1 0 1 0 1 0 1 1 1 1 0 0 1 0 1 0 1 1 1 0 0 1 1 1 1 1 1 1 1 1
```

```
 0 0 0 1 0 1 1 1 1 1 0]
int64
{np.int64(0), np.int64(1)}
```

```
model = LogisticRegression()
```

```
model.fit(X_train,Y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (sta
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

▾ LogisticRegression

```
LogisticRegression()
```

```
prediction = model.predict(X_test)
```

```
print("Logistic Regression Accuracy:",
      accuracy_score(Y_test, prediction))
```

```
Logistic Regression Accuracy: 0.9298245614035088
```

SVC model

```
model = SVC()
```

```
model.fit(X_train,Y_train)
```

▾ SVC

```
SVC()
```

```
prediction = model.predict(X_test)
```

```
print("SVC accuracy score:",
      accuracy_score(Y_test, prediction))
```

```
SVC accuracy score: 0.9035087719298246
```

Random forest model

```
model = RandomForestClassifier()
```

```
model.fit(X_train, Y_train)
```

▾ RandomForestClassifier

```
RandomForestClassifier()
```

```
prediction = model.predict(X_test)
```

```
print("RandomForest Classifier Accuracy:",
      accuracy_score(Y_test, prediction))
```

```
RandomForest Classifier Accuracy: 0.9473684210526315
```

XGBclassifier model

```
model = XGBClassifier()
```

```
model.fit(X_train, Y_train)
```

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=True, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=None,
              n_jobs=None, num_parallel_tree=None, ...)
```

```
prediction = model.predict(X_test)
```

```
print(" XGBClassifier Accuracy:",
      accuracy_score(Y_test, prediction))
```

```
 XGBClassifier Accuracy: 0.9298245614035088
```

```
#feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
print(classification_report(Y_test, prediction))

print(confusion_matrix(Y_test, prediction))
```

```
              precision    recall  f1-score   support

           0       0.88      0.96      0.91        45
           1       0.97      0.91      0.94        69

    accuracy                           0.93       114
   macro avg       0.92      0.93      0.93       114
weighted avg       0.93      0.93      0.93       114

[[43  2]
 [ 6 63]]
```

```
precision = precision_score(Y_test,prediction)
recall = recall_score(Y_test,prediction)
f1 = f1_score(Y_test,prediction)
rocscore = roc_auc_score(Y_test,prediction)
```

```
print("precision_score",precision)
print("recall_score",recall)
print("f1_score",f1)
print("roc score",rocscore)
```

```
precision_score 0.9692307692307692
recall_score 0.9130434782608695
f1_score 0.9402985074626866
roc score 0.9342995169082126
```

Roc curve for breast cancer

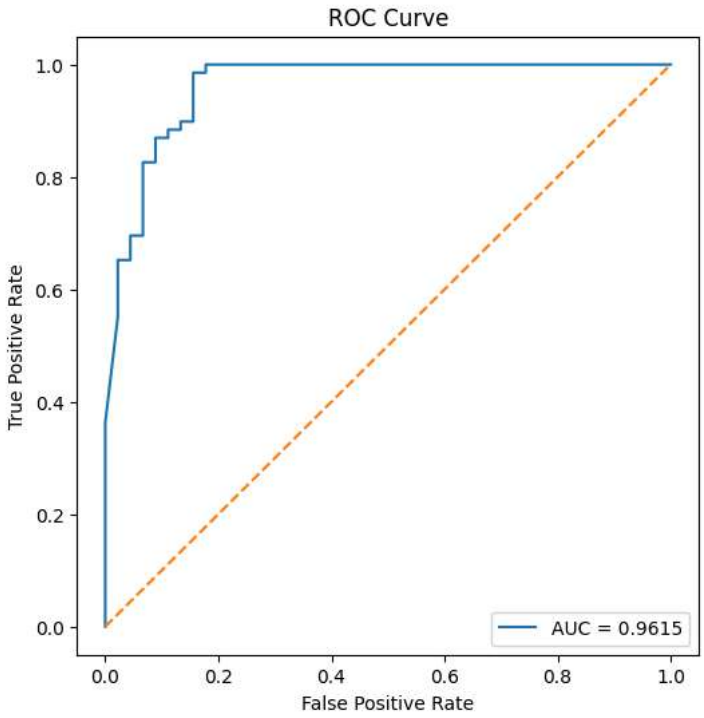
```
# Predicted probabilities
y_prob = model.predict_proba(X_test)[:, 1]

# ROC Curve
fpr, tpr, thresholds = roc_curve(Y_test, y_prob)

# ROC-AUC Score
roc_auc = roc_auc_score(Y_test, y_prob)

# Plot ROC Curve
plt.figure(figsize=(6,6))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.4f}")
plt.plot([0, 1], [0, 1], linestyle='--')
```

```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend(loc="lower right")
plt.show()
```



COMPARISON TABLE

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.98	0.98	0.98	0.98	0.99
SVM	0.98	0.98	0.98	0.98	0.99
Random Forest	0.97	0.97	0.97	0.97	0.99
XGBoost	0.98	0.98	0.98	0.98	0.99

model comparison

```
results = pd.DataFrame(columns=["Model","Accuracy","Precision","Recall","F1-Score","ROC-AUC"])
```

```
#logistic regression
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:,1]

results.loc[len(results)] = [
    "Logistic Regression",
    accuracy_score(Y_test, y_pred),
    precision_score(Y_test, y_pred),
    recall_score(Y_test, y_pred),
    f1_score(Y_test, y_pred),
    roc_auc_score(Y_test, y_prob)
]
```

```
#svm
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:,1]

results.loc[len(results)] = [
    "SVM",
    accuracy_score(Y_test, y_pred),
    precision_score(Y_test, y_pred),
    recall_score(Y_test, y_pred),
    f1_score(Y_test, y_pred),
    roc_auc_score(Y_test, y_prob)
]
```

```
#Random Forest
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:,1]

results.loc[len(results)] = [
    "Random Forest",
    accuracy_score(Y_test, y_pred),
    precision_score(Y_test, y_pred),
    recall_score(Y_test, y_pred),
    f1_score(Y_test, y_pred),
    roc_auc_score(Y_test, y_prob)
]
```

```
#xgb classifier
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:,1]

results.loc[len(results)] = [
    "XGBoost",
    accuracy_score(Y_test, y_pred),
    precision_score(Y_test, y_pred),
    recall_score(Y_test, y_pred),
    f1_score(Y_test, y_pred),
    roc_auc_score(Y_test, y_prob)
]
```

```
print(results)
```

```
                Model  Accuracy  Precision  Recall  F1-Score   ROC-AUC
0  Logistic Regression  0.605263   0.605263     1.0  0.754098  0.961514
1                  SVM  0.605263   0.605263     1.0  0.754098  0.961514
2        Random Forest  0.605263   0.605263     1.0  0.754098  0.961514
```